

ÉCOLE POLYTECHNIQUE DE THIÈS

Département Génie Informatique et Télécommunications

Master Ingénierie Logicielle et Intelligence Artificielle

Première année

SUVAC — Kaay Ñakku

Plateforme de suivi de la vaccination pré et post natale
avec rappels vocaux en wolof

Module : Développement Web Backend

Réalisé par :

Fallou DIOUCK

Encadré par :

Professeur Michel SECK

Code source :
`https://github.com/Fallou236/suvac`

Année universitaire 2026 — 2027

Résumé

SUVAC est une plateforme web de suivi de la vaccination pré et post natale conçue pour les postes de santé sénégalais. Elle calcule le calendrier vaccinal de chaque bénéficiaire à partir du schéma du Programme Élargi de Vaccination, affiche à l'agent la file des personnes attendues chaque matin, et envoie aux mères un rappel avant chaque échéance.

La particularité du système tient à la forme de ce rappel. Au Sénégal, une part importante des femmes en âge de procréer ne lit pas couramment le français. Un message écrit ne les atteint pas. SUVAC leur adresse donc un *message vocal en wolof*, synthétisé à partir du texte composé par le système, encapsulé dans une vidéo et remis par WhatsApp.

Le travail a été mené en trois sprints sur un dépôt Git public, avec intégration continue, tests automatisés et décisions d'architecture documentées. Au terme du projet, 286 tests couvrent 87% du code serveur et l'interface atteint 85,6% de couverture. La chaîne complète a été vérifiée de bout en bout : un rappel vaccinal composé en wolof, prononcé par synthèse vocale, est parvenu sur un téléphone sénégalais.

Mots-clés : santé maternelle et infantile, vaccination, wolof, synthèse vocale, Django, React, WhatsApp Business API.

Table des matières

Résumé **1**

1 Introduction **4**
1.1 Contexte . . . 4
1.2 Problème . . . 4
1.3 Objectifs . . . 4
1.4 Objectifs pédagogiques . . . 5
1.5 Périmètre . . . 5
1.6 Méthode de travail . . . 5

2 Analyse des besoins **6**
2.1 Acteurs du système . . . 6
2.2 Diagramme de cas d'utilisation . . . 7
2.3 Description textuelle des cas principaux . . . 8
2.3.1 Administrer une dose . . . 8
2.3.2 Envoyer un rappel vocal . . . 9
2.4 Exigences fonctionnelles . . . 9
2.5 Règles de gestion . . . 10
2.6 Exigences non fonctionnelles . . . 10

3 Conception **11**
3.1 Modèle du domaine . . . 11
3.1.1 Choix structurants . . . 14
3.2 Diagrammes de séquence système . . . 14
3.3 Diagramme d'interaction — envoi d'un rappel . . . 15
3.4 Diagramme des classes participantes . . . 17
3.5 Passage au modèle relationnel . . . 17
3.5.1 Règle 1 — cardinalité (?..1) d'un côté . . . 17
3.5.2 Règle 2 — cardinalité (?..N) des deux côtés . . . 18
3.5.3 Règle 3 — généralisation . . . 18
3.6 Modèle physique . . . 18

4 Architecture **20**
4.1 Vue d'ensemble . . . 20
4.2 Diagramme de composants . . . 21
4.3 Diagramme de déploiement . . . 22
4.4 Organisation du code . . . 22
4.5 Contrat d'API typé . . . 23

5 Réalisation **24**
5.1 Choix technologiques . . . 24
5.2 Le moteur de calendrier . . . 24
5.2.1 Les cinq statuts . . . 24
5.2.2 Le recalcul après une dose . . . 24
5.3 La composition des messages wolof . . . 25
5.3.1 Ce que dit le message . . . 25
5.3.2 Deux garde-fous . . . 25

5.4 La production du message vocal . . . 25
5.4.1 Deux mécanismes . . . 25
5.4.2 Le coût de la synthèse et sa parade . . . 26
5.5 L'encapsulation vidéo . . . 26
5.6 Sécurité . . . 26
5.6.1 Le cloisonnement par poste . . . 26
5.6.2 Le mot de passe initial . . . 26
5.6.3 L'invalidation des sessions . . . 26
5.6.4 Le journal d'audit . . . 27

6 Qualité et industrialisation 28
6.1 Stratégie de tests . . . 28
6.1.1 Ce que les tests ont attrapé . . . 28
6.2 Intégration continue . . . 28
6.3 Décisions d'architecture . . . 29
6.4 Volumétrie . . . 29

7 Résultats 30
7.1 Authentification . . . 30
7.2 Le poste de travail de l'agent . . . 30
7.2.1 La file du jour . . . 30
7.2.2 Le carnet d'un bénéficiaire . . . 31
7.2.3 La saisie d'une dose . . . 32
7.3 Le pilotage du superviseur . . . 32
7.4 Le journal des rappels . . . 33
7.5 L'espace de la bénéficiaire . . . 34
7.6 L'administration . . . 34
7.7 Le rappel reçu . . . 35

8 Bilan et perspectives 37
8.1 Ce qui a été livré . . . 37
8.2 Ce qui n'a pas été fait . . . 37
8.2.1 Le mode hors ligne . . . 37
8.2.2 Le code à usage unique . . . 37
8.2.3 L'évaluation de la compréhension . . . 37
8.3 Ce que le projet a appris . . . 37
8.3.1 Les contraintes externes dictent la conception . . . 37
8.3.2 Les tests révèlent ce qu'on ne cherchait pas . . . 38
8.3.3 Documenter les décisions a une valeur propre . . . 38
8.4 Perspectives . . . 38
8.5 Conclusion . . . 38

A Installation et vérification 39
A.1 Prise en main rapide . . . 39
A.2 Comptes de démonstration . . . 39
A.3 Vérification . . . 39

B Déclencher un balayage 40

Chapitre 1

Introduction

1.1 Contexte

La vaccination infantile est l'une des interventions de santé publique au meilleur rapport coût-efficacité. Au Sénégal, le Programme Élargi de Vaccination définit un calendrier précis : neuf vaccins, administrés selon des échéances calculées à partir de la date de naissance, certaines doses devant respecter un intervalle minimal entre elles.

Ce calendrier est consigné dans un carnet papier remis à la mère. Ce carnet présente deux faiblesses. Il se perd, et sa perte emporte avec elle l'historique vaccinal de l'enfant. Surtout, il ne rappelle rien : c'est à la mère de se souvenir de la date du prochain rendez-vous, souvent plusieurs semaines à l'avance.

La conséquence est connue des agents de santé : l'abandon en cours de série. Un enfant reçoit sa première dose de pentavalent, puis la deuxième n'est jamais administrée. Les doses manquées ne sont pas détectées, parce que rien ne signale l'absence — un bénéficiaire qui ne vient pas n'apparaît nulle part.

1.2 Problème

Deux obstacles séparent le système de santé de la mère.

Le premier est celui de l'information : personne ne sait, un matin donné, quels enfants devraient se présenter au poste. Le registre est une liste chronologique des passages, pas un calendrier des échéances à venir.

Le second est celui de la langue. Les outils numériques de santé s'adressent en français à une population dont une part importante ne le lit pas. Selon les données de l'ANSD, le taux d'alphabétisation des femmes en milieu rural reste nettement inférieur à celui des hommes, et le français n'est pas la langue de la vie quotidienne dans la plupart des foyers. Un rappel par SMS écrit en français n'atteint pas sa destinataire.

1.3 Objectifs

L'objectif général est de **réduire les doses manquées en rendant visible ce qui est dû et en prévenant la bénéficiaire dans une langue qu'elle comprend**.

Il se décline en quatre objectifs spécifiques :

OS-1. Calculer et tenir à jour le calendrier vaccinal de chaque bénéficiaire à partir du schéma officiel, y compris après une dose manquée ou administrée hors délai.

OS-2. Donner à l'agent de santé une vue quotidienne des personnes attendues et de celles en retard.

OS-3. Prévenir les mères avant chaque échéance par un message vocal en wolof, acheminé par un canal qu'elles utilisent déjà.

OS-4. Donner au superviseur les indicateurs de couverture et d'acheminement nécessaires au pilotage de son poste.

1.4 Objectifs pédagogiques

Au-delà du produit, le projet visait à mettre en œuvre les pratiques enseignées dans le cursus : conception orientée objet avec UML, API REST documentée, tests automatisés, intégration continue, et traçabilité des décisions d'architecture.

1.5 Périmètre

Dans le périmètre

Le suivi vaccinal des enfants de zéro à cinq ans et des femmes enceintes ; le moteur de calendrier ; les rappels par WhatsApp et leur production vocale ; un espace de consultation pour les mères ; le pilotage par poste ; l'administration des comptes et des postes.

Hors périmètre

La gestion des stocks de vaccins, la chaîne du froid, la facturation, et l'interopérabilité avec le système national d'information sanitaire. Le mode hors ligne, bien que spécifié, n'a pas été réalisé dans le temps imparti ; il est traité au chapitre des perspectives.

1.6 Méthode de travail

Le développement a suivi un cycle itératif, proche du cycle en spirale : le cahier des charges a été découpé en trois incréments fonctionnels, chacun traité comme un sprint avec sa planification, sa réalisation et sa rétrospective.

Sprint	Thème	Contenu
S-1	Socle métier	Modèle de données, moteur de calendrier, API REST, cloisonnement par poste
S-2	Interface	Application React, écrans de l'agent, tableau de bord du superviseur, espace bénéficiaire
S-3	Communication	Planification des rappels, composition wolof, synthèse vocale, canal WhatsApp, administration

Table 1.1 – Découpage en sprints

Chaque sprint a donné lieu à un avenant au cahier des charges, et les décisions structurantes ont été consignées dans des fiches ADR (*Architecture Decision Records*) — neuf au total, référencées tout au long de ce rapport.

Chapitre 2

Analyse des besoins

2.1 Acteurs du système

Conformément à la définition d'UML, un acteur correspond à un rôle et non à une personne physique : la même personne peut être agent un jour et superviseur un autre.

Acteur	Rôle	Type
Agent de santé	Infirmier, sage-femme ou matrone du poste. Enregistre les bénéficiaires, administre les doses, ouvre les accès aux mères. Utilisateur principal du système.	Principal
Superviseur	Responsable du suivi de la couverture sur son poste. Consulte les indicateurs et le journal des rappels, gère le personnel. N'administre pas de doses.	Principal
Administrateur	Membre de l'équipe technique. Crée les postes, gère les comptes de tous les postes, consulte le journal d'audit.	Principal
Mère ou tutrice	Bénéficiaire finale. Reçoit les rappels vocaux, consulte le carnet de ses enfants si un accès lui est ouvert. Peut être non alphabétisée.	Principal
WhatsApp Cloud API	Plateforme de messagerie de Meta. Achemine les rappels et renvoie les accusés de réception.	Secondaire
Service de synthèse	Produit le message vocal wolof à partir du texte composé par le système.	Secondaire

Table 2.1 – Acteurs du système

Le superviseur *spécialise* l'agent : il dispose de toutes ses consultations, auxquelles s'ajoutent le pilotage et la gestion du personnel. Cette relation de généralisation apparaît sur le diagramme de cas d'utilisation.

2.2 Diagramme de cas d'utilisation

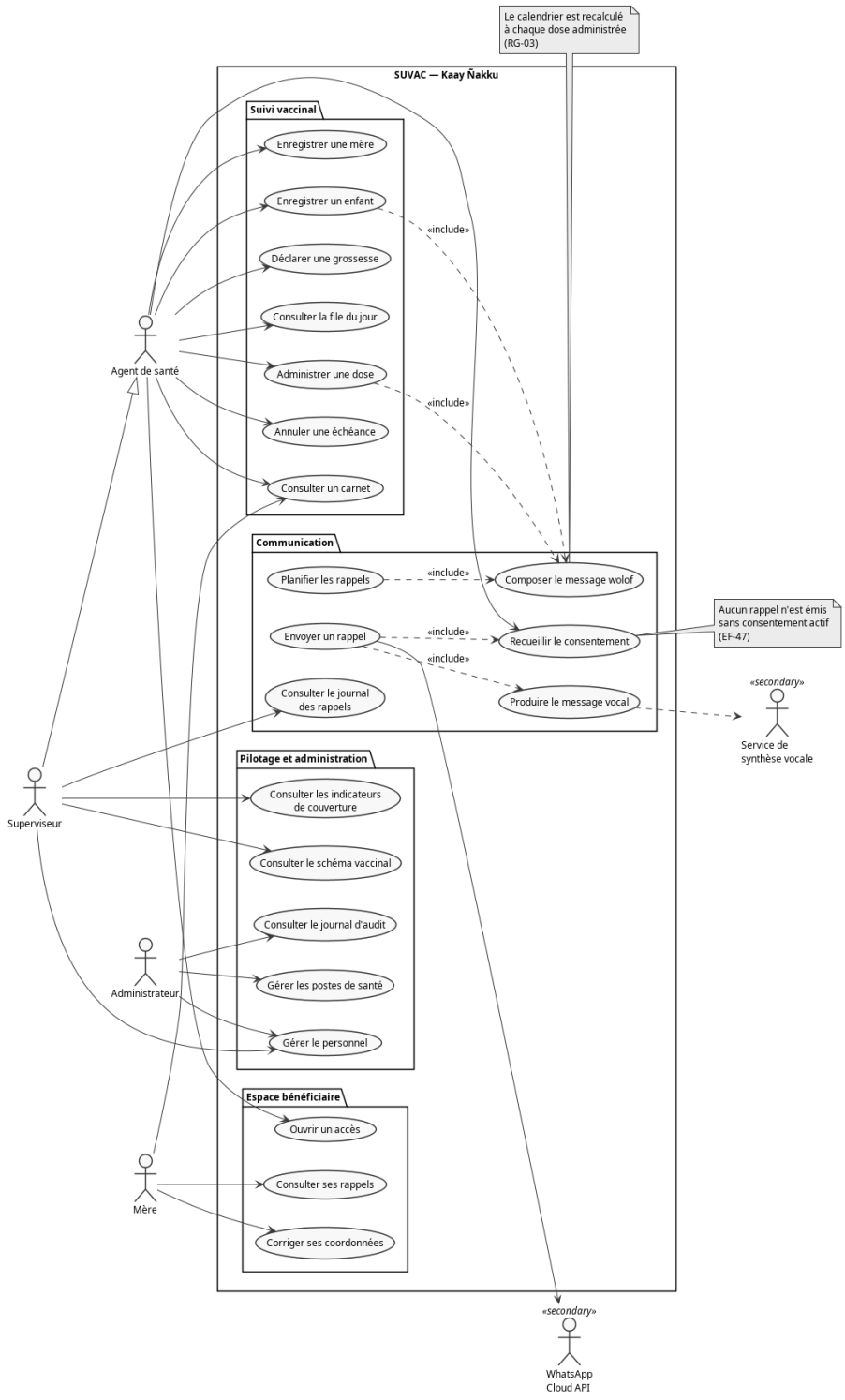


Figure 2.1 – Diagramme de cas d'utilisation du système SUVAC

Les cas sont regroupés en quatre paquetages correspondant aux domaines fonctionnels. Deux relations méritent d'être soulignées.

La relation `«include»` entre « administrer une dose » et « composer le message wolof » traduit une règle métier forte : le calendrier est recalculé à chaque dose enregistrée. Une dose administrée en retard décale les suivantes, et le système doit en tenir compte immédiatement.

La relation «include» entre « envoyer un rappel » et « recueillir le consentement » matérialise une contrainte réglementaire : aucun message n'est émis vers une bénéficiaire qui n'a pas consenti, et ce consentement est révocable à tout moment.

2.3 Description textuelle des cas principaux

2.3.1 Administrer une dose

Cas	Administrer une dose
Résumé	Enregistrement d'une dose de vaccin administrée à un bénéficiaire, et mise à jour du calendrier qui en découle
Acteur principal	Agent de santé
Pré-conditions	L'agent est authentifié ; le bénéficiaire est rattaché à son poste ; l'échéance existe et n'est pas déjà satisfaite
Résultat	Dose enregistrée, échéances suivantes recalculées, file du jour mise à jour
Description	1. L'agent consulte la file du jour 2. Il sélectionne un bénéficiaire 3. Le système affiche le carnet et les échéances 4. L'agent choisit l'échéance à satisfaire 5. Le système vérifie l'administrabilité 6. L'agent saisit la date et le numéro de lot 7. Le système enregistre la dose et recalcule les échéances suivantes
Exceptions	L'intervalle minimal depuis la dose précédente n'est pas respecté : le système refuse et explique le motif

Table 2.2 – Cas d'utilisation « Administrer une dose »

2.3.2 Envoyer un rappel vocal

Cas	Envoyer un rappel vocal
Résumé	Production et acheminement d'un message vocal wolof annonçant à une mère une échéance vaccinale
Acteur principal	Le système, déclenché par l'ordonnanceur
Acteurs secondaires	Service de synthèse vocale, WhatsApp Cloud API
Pré-conditions	La bénéficiaire a un consentement actif pour le canal ; elle dispose d'un numéro de téléphone
Résultat	Message remis, statut d'acheminement enregistré
Description	1. L'ordonnanceur déclenche le balayage quotidien 2. Le système relève les échéances dues, proches ou en retard 3. Il regroupe par bénéficiaire les échéances d'un même moment 4. Il compose le texte wolof correspondant 5. Il fait prononcer ce texte par le service de synthèse 6. Il encapsule l'audio dans une vidéo 7. Il téléverse la vidéo et envoie le modèle WhatsApp 8. Il enregistre le statut retourné
Exceptions	Service de synthèse indisponible : repli sur l'assemblage de segments pré-enregistrés Envoi refusé définitivement : le rappel est marqué en échec avec son motif, sans nouvelle tentative

Table 2.3 – Cas d'utilisation « Envoyer un rappel vocal »

2.4 Exigences fonctionnelles

Le cahier des charges recense une centaine d'exigences fonctionnelles. Les plus structurantes sont reprises ici ; le document complet figure en annexe du dépôt.

Réf.	**Exigence**	**Prio.**
EF-03	Un agent n'accède qu'aux bénéficiaires de son poste	Doit
EF-13	Chaque bénéficiaire dispose d'un identifiant public opaque	Doit
EF-20	Le système calcule les échéances à partir du schéma du PEV	Doit
EF-22	Le calendrier est recalculé à chaque dose administrée	Doit
EF-30	Une dose ne peut être administrée avant l'intervalle minimal	Doit
EF-40	Un balayage quotidien génère les rappels dus	Doit
EF-41	Un rappel est émis trois jours avant, le jour même, puis en relance tant que la dose reste rattrapable	Doit
EF-43	Le rappel WhatsApp contient un message vocal en wolof	Doit
EF-47	Aucun rappel n'est émis sans consentement actif	Doit
EF-90	Le superviseur crée les comptes du personnel de son poste	Doit
EF-91	Un compte créé ne peut rien faire avant changement du mot de passe initial	Doit
EF-100	Les échéances d'un même moment tiennent en un seul message	Doit
EF-101	Le message vocal est encapsulé dans une vidéo	Doit

Table 2.4 – Exigences fonctionnelles structurantes

2.5 Règles de gestion

Quatre règles méritent d'être citées, parce qu'elles ont façonné la conception.

RG-03 Toute dose administrée déclenche le recalcul des échéances suivantes de la même série.

RG-09 Une bénéficiaire ne reçoit qu'un seul message par moment de rappel, même si plusieurs de ses enfants ont une échéance le même jour.

RG-10 Un enregistrement de dose est immuable : une correction crée une nouvelle version et conserve la trace de la précédente.

RG-12 Un compte d'utilisateur n'est jamais supprimé, seulement désactivé, afin que les actes qu'il a enregistrés gardent leur auteur.

2.6 Exigences non fonctionnelles

Réf.	Exigence
ENF-02	Une page doit rester utilisable sur une connexion 2G
ENF-08	L'authentification repose sur des jetons signés, à durée limitée
ENF-16	Aucun secret n'est versionné : configuration par variables d'environnement
ENF-20	Le traitement respecte la loi sénégalaise n° 2008-12 sur la protection des données à caractère personnel
ENF-21	Le consentement est recueilli, horodaté et révocable
ENF-24	Les données de démonstration sont entièrement fictives
ENF-25	Une bénéficiaire n'accède qu'à son propre dossier
ENF-27	Un changement de mot de passe invalide les sessions ouvertes
ENF-28	Les actes d'administration sont journalisés en écriture seule

Table 2.5 – Exigences non fonctionnelles

Chapitre 3

Conception

La conception a suivi la méthode minimale présentée en cours : des besoins aux cas d'utilisation, des cas aux séquences système, du modèle du domaine aux classes participantes, puis aux classes de conception et au code.

3.1 Modèle du domaine

Le modèle du domaine décrit les concepts invariants, indépendamment des technologies employées. Aucune opération n'y figure : il s'agit de nommer les choses et leurs liens.

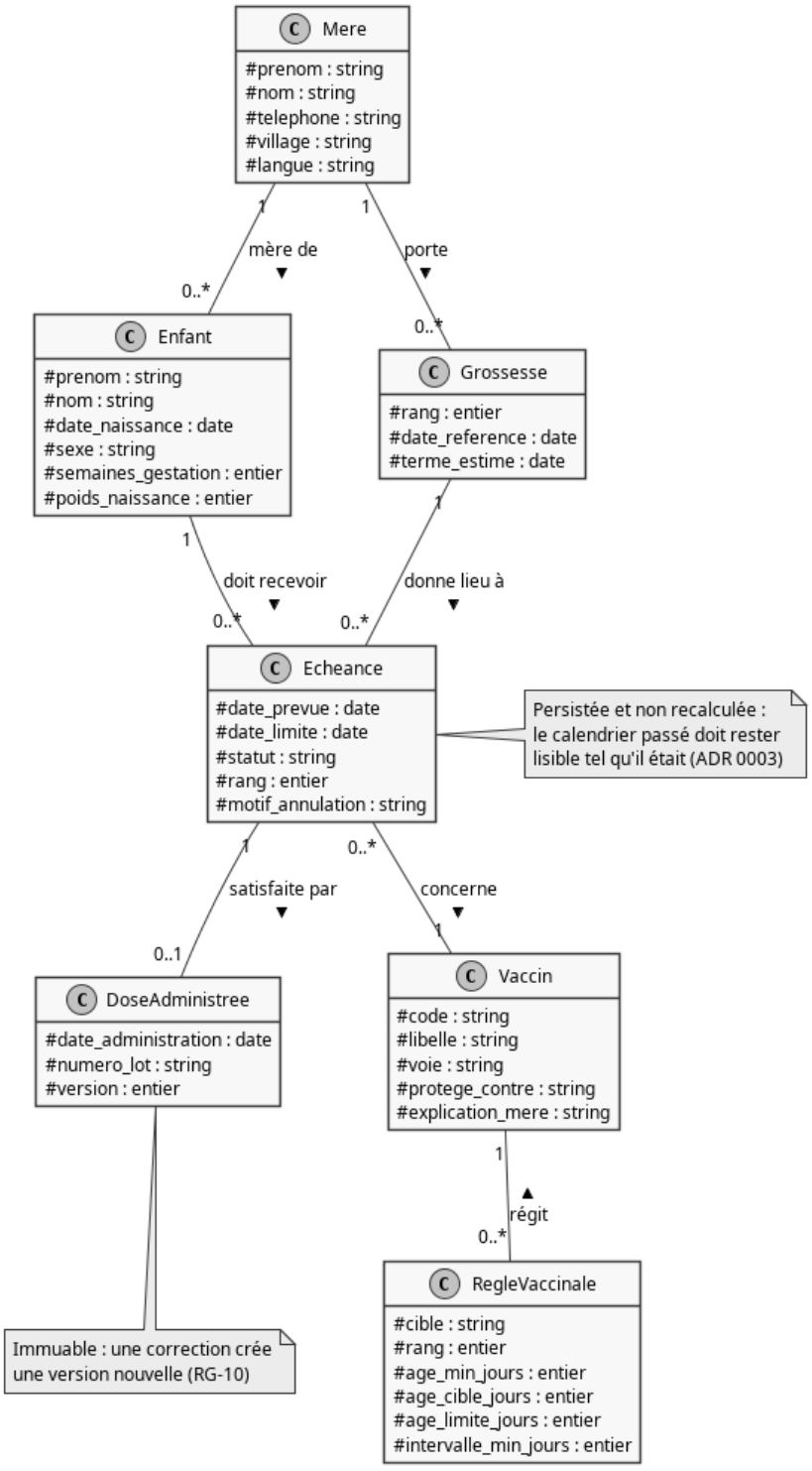


Figure 3.1 – Modèle du domaine — le suivi vaccinal

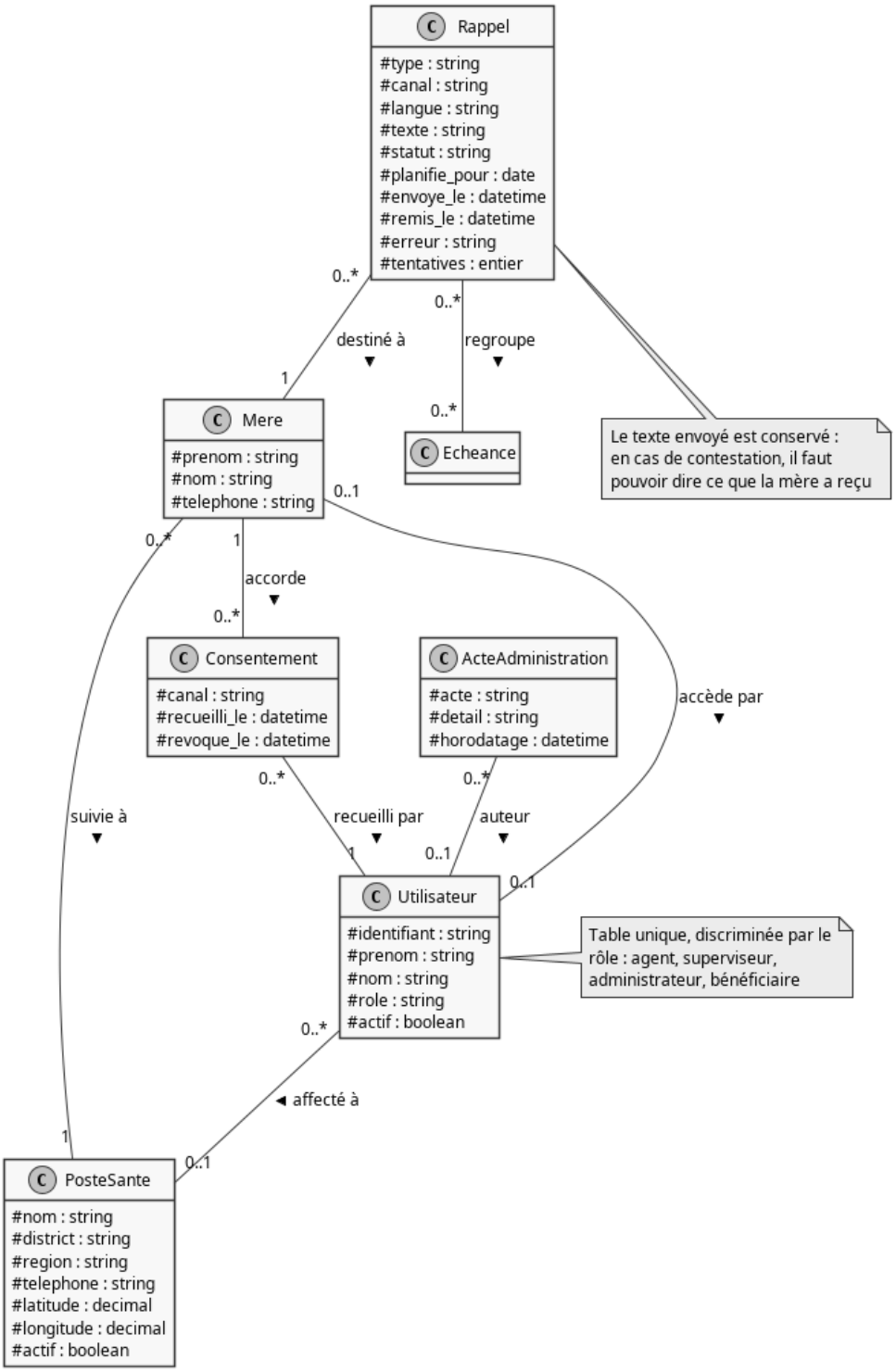


Figure 3.2 – Modèle du domaine — accès et communication

3.1.1 Choix structurants

La séparation mère / enfant / grossesse

Une mère peut avoir plusieurs enfants suivis et plusieurs grossesses successives. Les trois entités sont distinctes parce que leurs calendriers vaccinaux sont différents : un enfant suit le schéma infantile, une femme enceinte reçoit le vaccin antitétanique selon un schéma propre.

La classe `Echeance` porte donc deux associations optionnelles — vers `Enfant` ou vers

`Grossesse` — dont exactement une est renseignée. Cette contrainte est garantie en base par une contrainte de vérification.

L'échéance comme entité de plein droit

Le choix le plus discuté a été de *persister* les échéances plutôt que de les recalculer à la volée à chaque consultation.

Recalculer serait plus simple et éviterait toute désynchronisation. Mais le calendrier passé doit rester lisible tel qu'il était : si le schéma vaccinal évolue, un enfant suivi sous l'ancien schéma ne doit pas voir son historique réécrit. Une échéance annulée pour contre-indication doit rester visible avec son motif. Un recalcul effacerait ces informations.

Cette décision est consignée dans l'ADR 0003. Son prix est la nécessité d'un rafraîchissement : une commande quotidienne met à jour les statuts en fonction de la date du jour.

L'immuabilité des doses

Une dose administrée n'est jamais modifiée. Une correction crée un nouvel enregistrement versionné, l'ancien étant conservé. Sur des données de santé, savoir ce qui a été enregistré et quand n'est pas un confort : c'est ce qui permet de répondre en cas de contestation.

C'est aussi pourquoi un compte d'utilisateur n'est jamais supprimé : chaque dose porte l'identité de l'agent qui l'a enregistrée, et cette trace doit rester lisible.

3.2 Diagrammes de séquence système

Les séquences système formalisent la description textuelle des cas d'utilisation. Le système y est vu comme une boîte noire : seules les interactions avec les acteurs apparaissent.

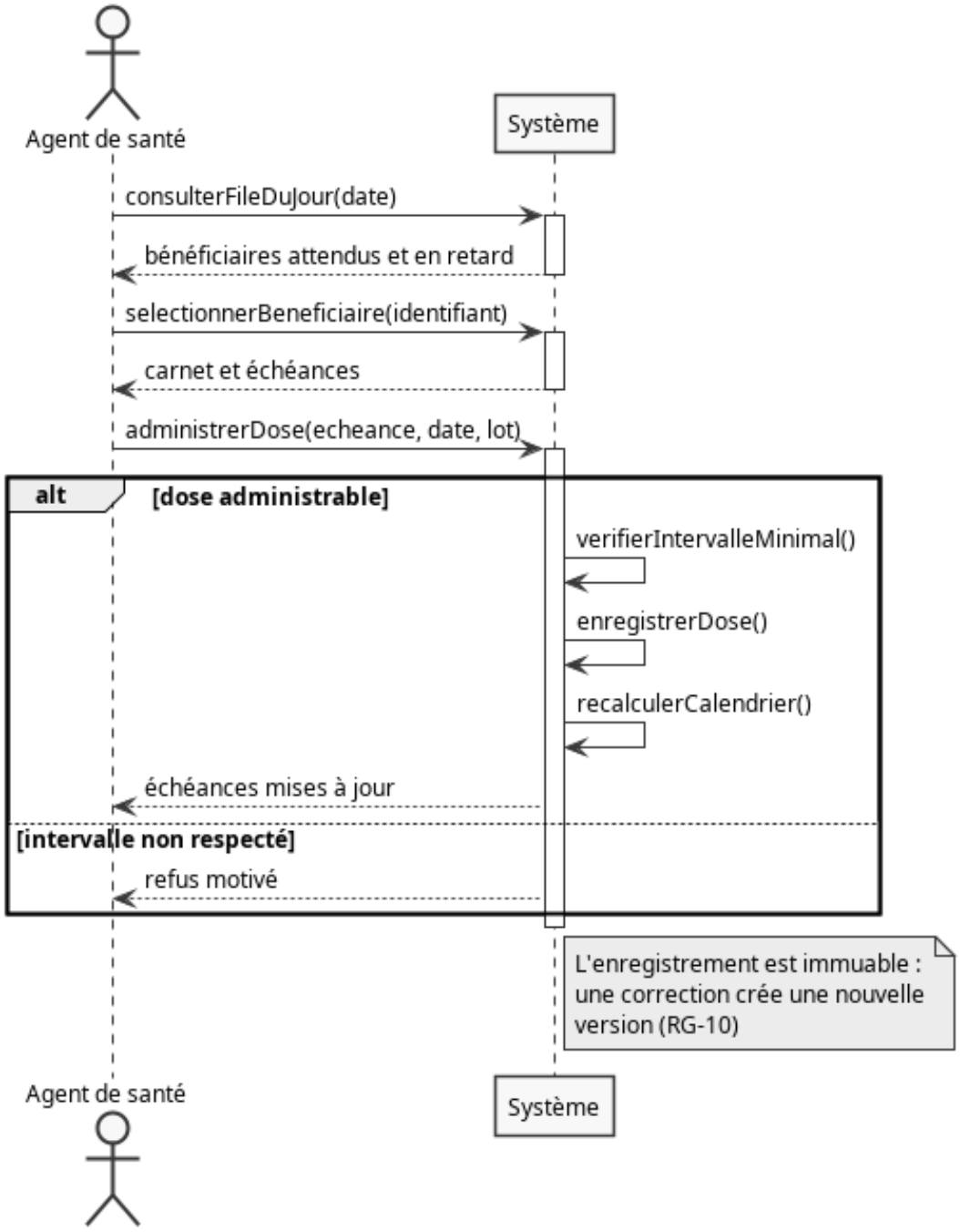


Figure 3.3 – Séquence système — administration d'une dose

Ce diagramme met en évidence l'opération système `administrerDose()`, et la vérification qui la précède. Le fragment `alt` traduit l'exception identifiée dans la description textuelle : le refus motivé quand l'intervalle minimal n'est pas respecté.

3.3 Diagramme d'interaction — envoi d'un rappel

Le second diagramme de séquence adopte une vue boîte blanche : le système est éclaté en ses composants internes, et l'on voit comment ils coopèrent pour produire et acheminer un rappel.

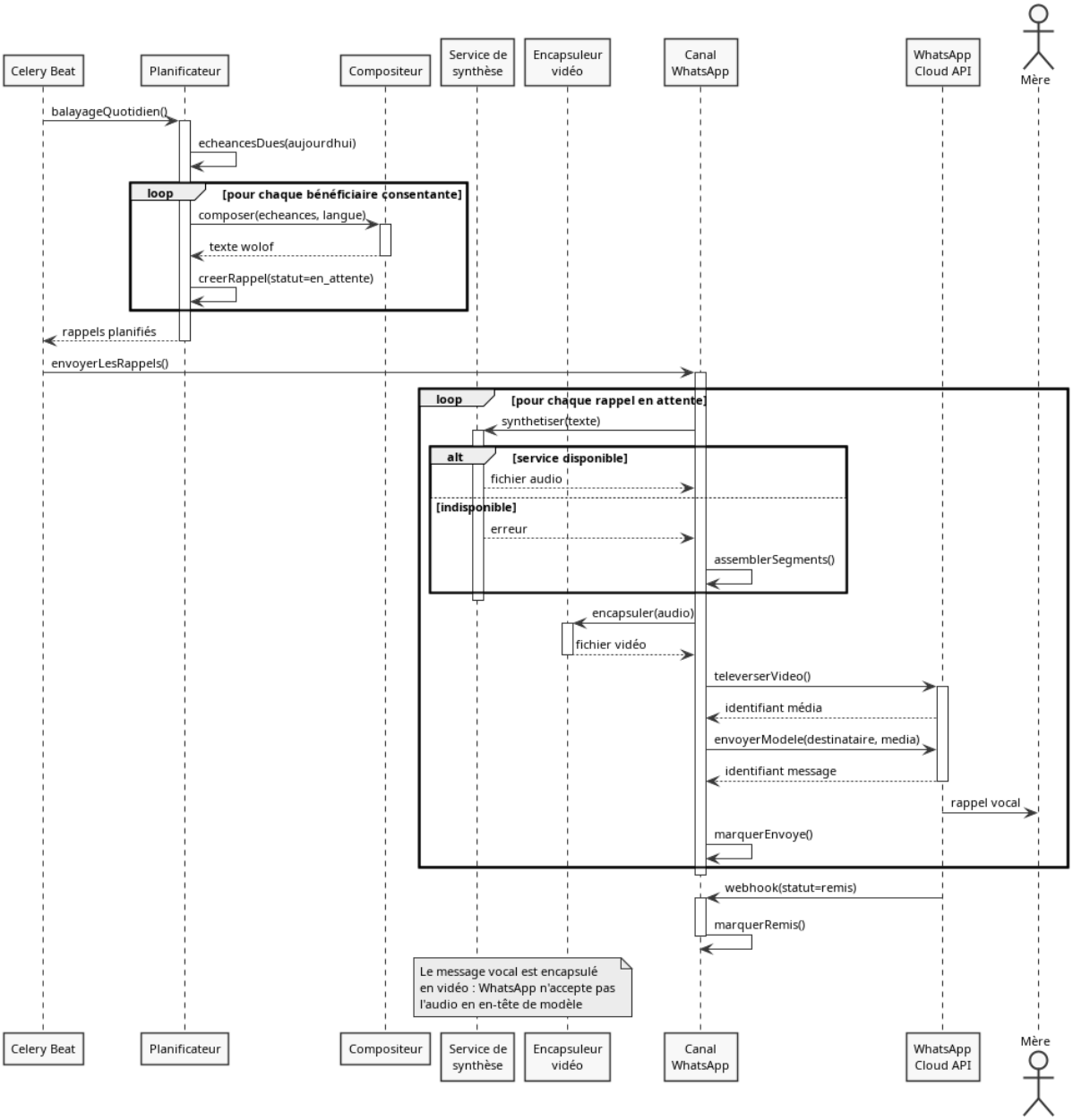


Figure 3.4 – Diagramme d'interaction — production et envoi d'un rappel vocal

Trois points de conception apparaissent sur ce diagramme.

Le regroupement par bénéficiaire. La boucle de planification itère sur les bénéficiaires, pas sur les échéances. C'est la traduction directe de RG-09 : une mère dont trois enfants ont une échéance le même jour reçoit un seul message, qui les mentionne tous.

Le repli sur les segments. Le fragment `alt` autour de l'appel au service de synthèse exprime une décision : l'indisponibilité du service ne doit pas empêcher l'envoi. Mieux vaut un message assemblé à partir d'enregistrements qu'une mère non prévenue.

Le webhook asynchrone. L'accusé de réception ne revient pas dans le flux d'envoi : il arrive plus tard, par un appel entrant de Meta. Le statut du rappel évolue donc après coup, ce qui impose de le persister plutôt que de le déduire.

3.4 Diagramme des classes participantes

Le diagramme des classes participantes enrichit le modèle du domaine des classes de dialogue et de contrôle. Il est présenté ici pour le cas « administrer une dose ».

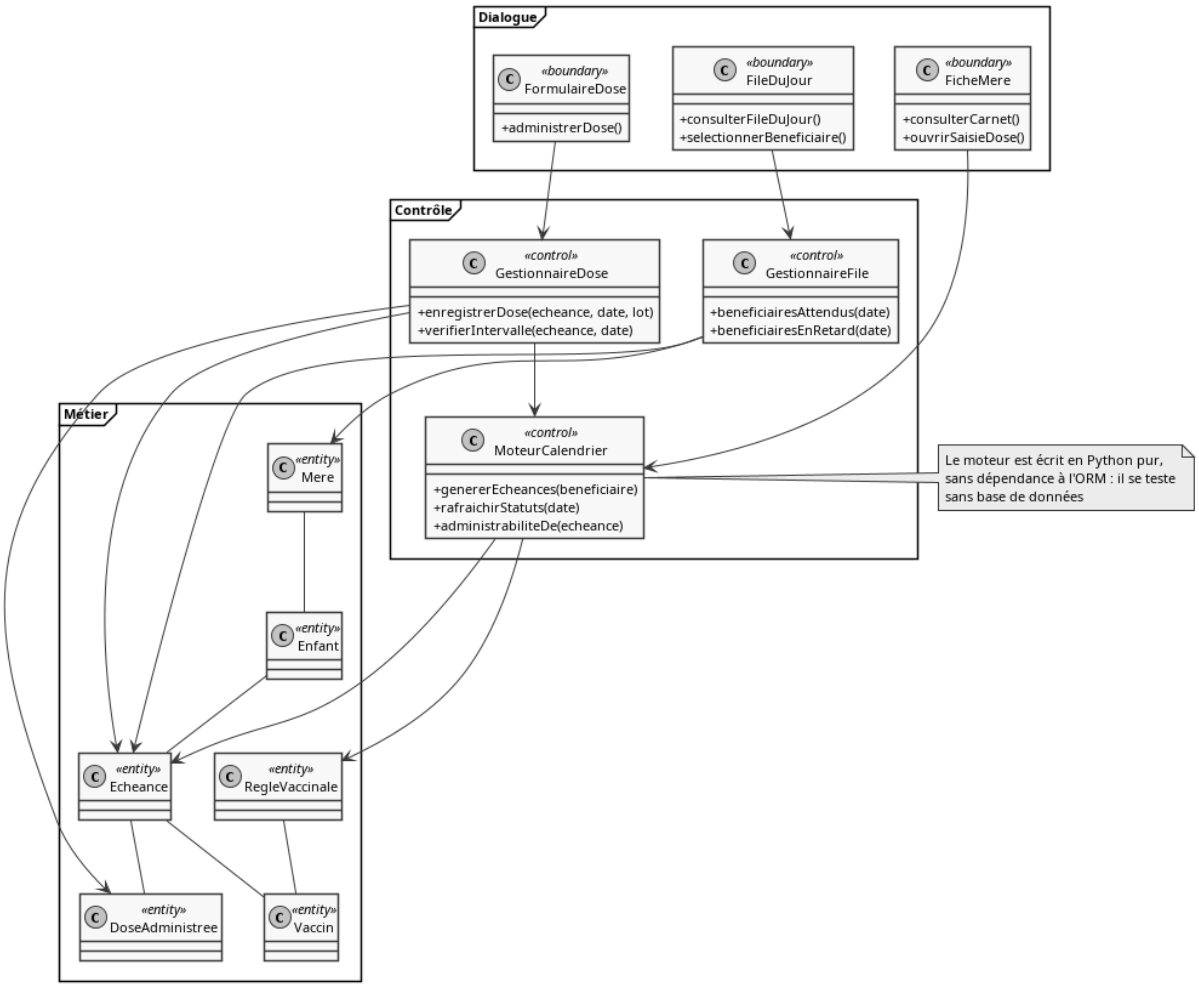


Figure 3.5 – Classes participantes du cas « administrer une dose »

L'architecture en couches est respectée : les classes de dialogue ne communiquent qu'avec des contrôles, les classes métier également, et les contrôles font le lien entre les deux.

La classe `MoteurCalendrier` mérite un commentaire. Elle concentre toute la logique de calcul des échéances, et elle est écrite en Python pur, sans aucune dépendance à l'ORM de Django. Ce choix a une conséquence pratique importante : le moteur se teste sans base de données, ce qui rend ses tests instantanés et permet d'en écrire beaucoup.

3.5 Passage au modèle relationnel

Le diagramme de classes se traduit en modèle relationnel par application des règles vues en cours.

3.5.1 Règle 1 — cardinalité (?..1) d'un côté

L'association entre `Mere` et `Enfant` porte la cardinalité 1 du côté de la mère. L'identifiant de `Mere` devient donc clé étrangère dans la table `Enfant` :

```
Mere(id, prenom, nom, telephone, village, langue, poste_id, compte_id)
Enfant(id, prenom, nom, date_naissance, sexe, mere_id, poste_id, ...)

Contrainte : Enfant.mere_id reference Mere.id
```

Le même raisonnement s'applique aux associations `Echeance`–`Vaccin`, `DoseAdministree`–`Echeance` et `Utilisateur`–`PosteSante`.

3.5.2 Règle 2 — cardinalité (?..N) des deux côtés

L'association entre `Rappel` et `Echeance` est de type plusieurs-à-plusieurs : un rappel peut regrouper plusieurs échéances, et une échéance peut donner lieu à plusieurs rappels successifs (préavis, jour même, relance). Elle se traduit par une table d'association :

```
Rappel(id, mere_id, type, canal, langue, texte, statut, ...)
Echeance(id, enfant_id, grossesse_id, vaccin_id, date_prevue, ...)
RappelEcheance(rappel_id, echeance_id)

Contraintes : RappelEcheance.rappel_id reference Rappel.id
              RappelEcheance.echeance_id reference Echeance.id
```

3.5.3 Règle 3 — généralisation

Le modèle comporte une généralisation implicite : les rôles d'agent, de superviseur, d'administrateur et de bénéficiaire sont des spécialisations d'`Utilisateur`.

Des trois méthodes présentées en cours, c'est la **première** qui a été retenue : une table unique, avec un attribut discriminant.

```
Utilisateur(id, identifiant, prenom, nom, role, poste_id, actif, ...)
  role parmi {agent, superviseur, admin, beneficiaire}
```

Ce choix se justifie par la nature des rôles. Les quatre partagent la quasi totalité de leurs attributs ; seules leurs *permissions* diffèrent, et celles-ci relèvent du comportement, pas des données. Créer quatre tables aurait multiplié les jointures sans gain de structure. De plus, un superviseur peut être transféré ou rétrogradé : avec des tables séparées, un changement de rôle imposerait de déplacer la ligne, et les clés étrangères pointant vers elle deviendraient invalides.

3.6 Modèle physique

Le modèle physique compte dix-huit tables. Les principales sont représentées ci-dessous sous forme simplifiée.

Table	Rôle
`accounts_postesante`	Structures sanitaires de rattachement
`accounts_utilisateur`	Comptes, tous rôles confondus
`accounts_journalaudit`	Actes d'administration, en écriture seule
`beneficiaires_mere`	Mères et tutrices suivies
`beneficiaires_enfant`	Enfants rattachés à une mère
`beneficiaires_grossesse`	Grossesses déclarées
`beneficiaires_consentement`	Consentements, horodatés et révocables
`vaccination_vaccin`	Référentiel des vaccins
`vaccination_reglevaccinale`	Règles du schéma du PEV
`suivi_echeance`	Échéances calculées et persistées
`suivi_doseadministree`	Doses enregistrées, immuables
`rappels_rappel`	Rappels planifiés et leur acheminement

Table 3.1 – Principales tables du modèle physique

Identifiants publics

Chaque entité exposée par l'API porte, en plus de sa clé primaire interne, un identifiant public de type UUID. Les URL de l'API utilisent cet identifiant.

Ce choix répond à une préoccupation concrète. Avec des clés séquentielles, l'adresse `/api/meres/47/` révèle qu'il existe au moins quarante-sept mères enregistrées, et invite à essayer `/48/`. Le cloisonnement par poste protège déjà contre l'accès, mais un identifiant opaque supprime l'invitation elle-même.

Il règle aussi un problème que les homonymes rendent sensible : dans un même village, plusieurs femmes peuvent porter le même nom. Rien dans le système ne repose sur l'identité nominale.

Chapitre 4

Architecture

4.1 Vue d'ensemble

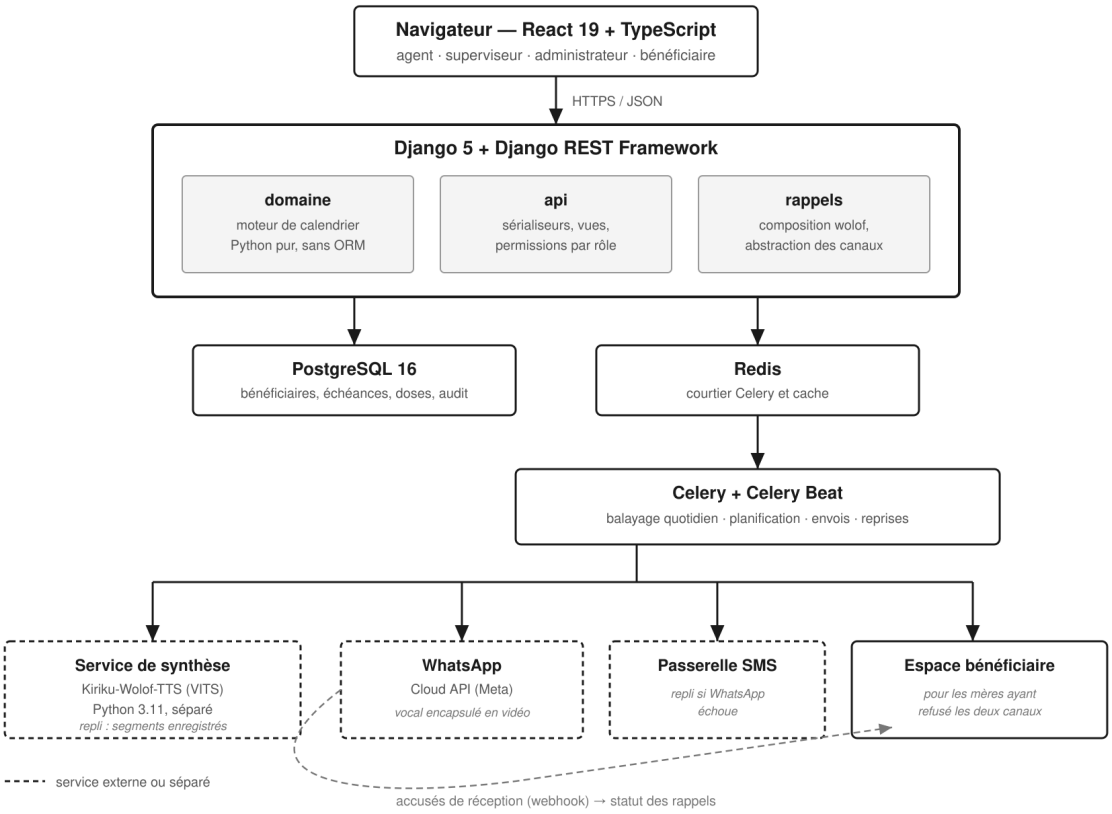


Figure 4.1 – Architecture générale du système

Le navigateur dialogue avec Django en HTTPS et JSON. Django porte le moteur de calendrier, l'API et l'abstraction des canaux de notification. PostgreSQL conserve les données, Redis sert de courtier à Celery, qui exécute le balayage quotidien et les envois.

Les messages vocaux passent par un service de synthèse séparé. WhatsApp est le canal principal, le SMS le repli, et l'application elle-même sert les mères qui ont refusé les deux.

4.2 Diagramme de composants

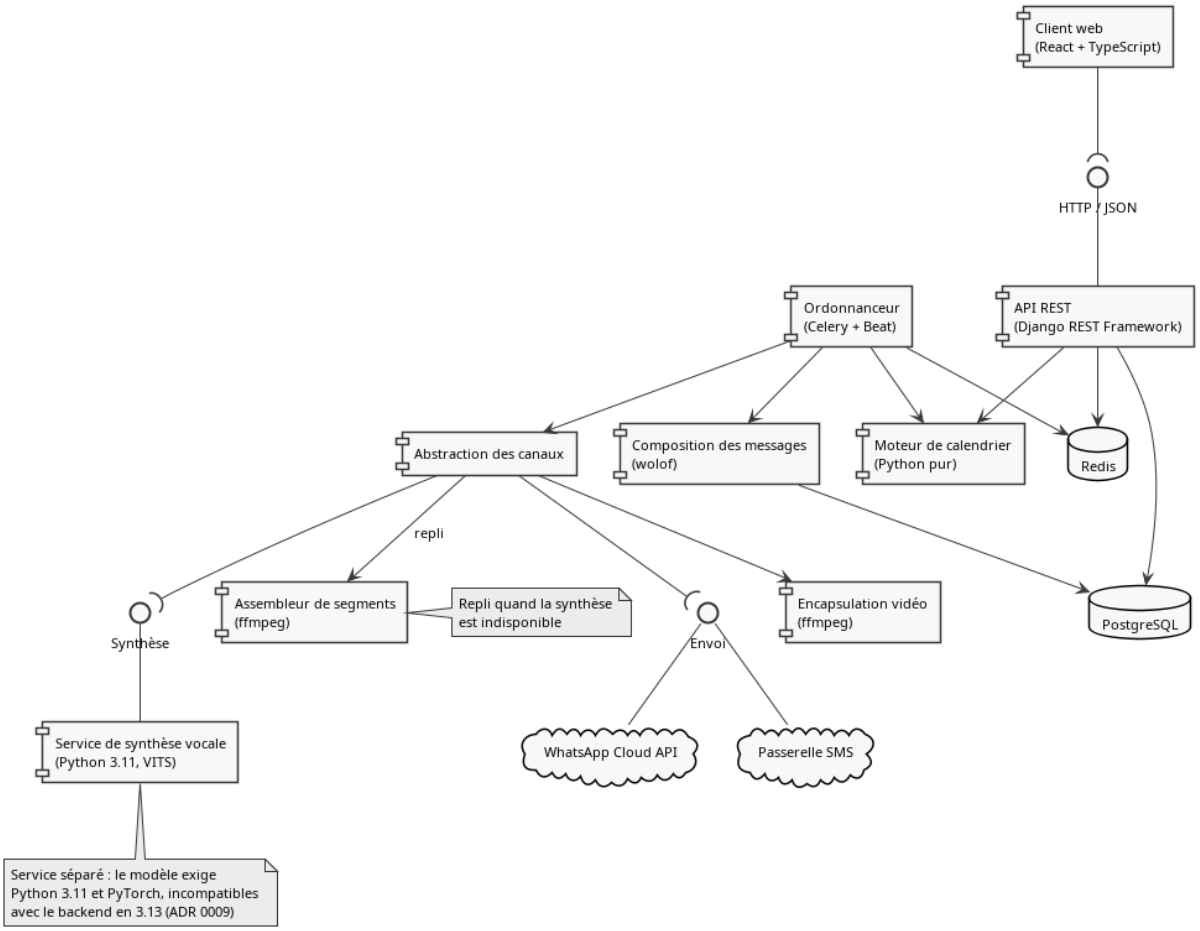


Figure 4.2 – Diagramme de composants

Le diagramme fait apparaître les interfaces requises et fournies. Deux points de conception s'y lisent.

L'abstraction des canaux

Le composant « abstraction des canaux » expose une interface unique `Envoi`, implémentée par WhatsApp, le SMS, un canal applicatif et un canal de simulation. Le code de planification ne connaît aucun de ces canaux : il demande l'envoi d'un message et reçoit un résultat.

Cette indirection a eu une conséquence heureuse pendant le développement : l'ensemble de la logique de rappel a pu être écrite et testée avec le canal de simulation, avant même que le compte WhatsApp ne soit ouvert.

La séparation du service de synthèse

Le service de synthèse vocale est un composant distinct, appelé en HTTP, et non une bibliothèque importée. Ce n'est pas un choix esthétique mais une contrainte subie, documentée dans l'ADR 0009 : le modèle de synthèse wolof exige Python 3.11, PyTorch et une version ancienne de la bibliothèque `transformers`, incompatibles avec le backend qui tourne en Python 3.13 avec Django 5.

L'alternative aurait été de rétrograder l'ensemble du projet. Le coût d'un service séparé — une pièce de plus à déployer — a paru moindre.

4.3 Diagramme de déploiement

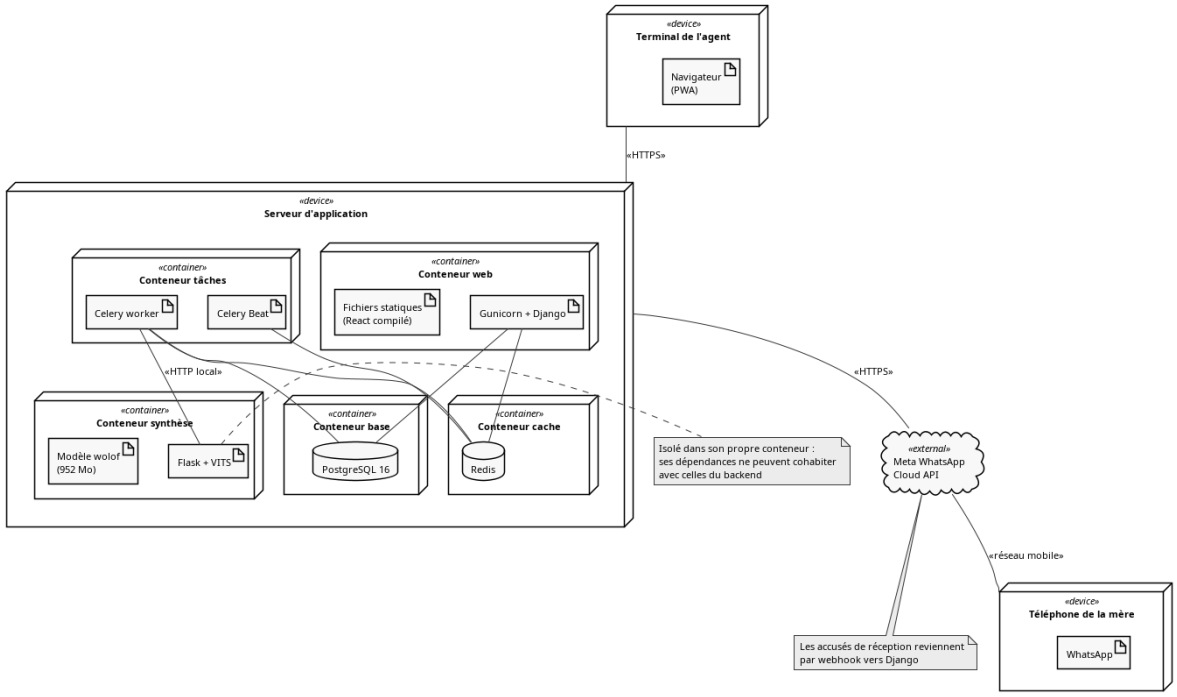


Figure 4.3 – Diagramme de déploiement

Chaque composant est isolé dans son conteneur. Cette séparation n'est pas seulement une commodité de développement : elle permet au service de synthèse d'avoir ses propres dépendances sans contaminer le reste.

4.4 Organisation du code

Le dépôt suit une séparation nette entre le serveur, le client et les services annexes.

```
suvac/
+-- backend/
|   +-- apps/
|   |   +-- commun/          # bases partagées, données de démonstration
|   |   +-- accounts/        # comptes, postes, audit
|   |   +-- domaine/         # moteur de calendrier (Python pur)
|   |   +-- vaccination/     # référentiel des vaccins et du schéma
|   |   +-- beneficiaires/   # mères, enfants, grossesses, consentements
|   |   +-- suivi/           # échéances et doses
|   |   +-- rappels/         # planification, composition, canaux
|   |   +-- pilotage/        # indicateurs de couverture
|   +-- config/              # configuration Django
+-- frontend/
|   +-- src/
|       +-- api/             # client HTTP et types générés
|       +-- composants/      # bibliothèque d'interface
|       +-- pages/           # écrans
|       +-- etat/            # état applicatif
+-- services/
|   +-- synthese/            # service de synthèse vocale
+-- docs/                    # cahier des charges, ADR, rapport
```

Le découpage en applications Django suit les domaines métier et non les couches techniques. L'application `domaine` fait exception : elle ne contient aucun modèle Django, seulement des fonctions de calcul pur.

4.5 Contrat d'API typé

Un point mérite d'être souligné parce qu'il a éliminé une classe entière d'erreurs.

Django REST Framework génère automatiquement un schéma OpenAPI à partir des sérialiseurs. Ce schéma est converti en types TypeScript par un générateur, et ces types sont utilisés par le client. Le contrat est donc vérifié à la compilation.

```
Sérialiseurs Django  -->  schéma OpenAPI  -->  types TypeScript  -->  client
    React
```

Conséquence concrète : renommer un champ côté serveur fait échouer la compilation du frontend. L'erreur apparaît à la construction et non à l'exécution, chez un agent de santé au milieu d'une consultation.

Chapitre 5

Réalisation

5.1 Choix technologiques

Couche	Technologie	Justification
Serveur	Django 5 + DRF	Maturité de l'ORM, administration intégrée, exigence du module
Base	PostgreSQL 16	Intégrité relationnelle, contraintes de vérification
Tâches	Celery + Beat	Planification du balayage et des envois
Courtier	Redis	Courtier Celery et cache applicatif
Client	React 19 + TypeScript	Exigence du module ; le typage sert directement la qualité
État serveur	TanStack Query	Cache, reprise sur erreur
Interface	Tailwind CSS	Jetons de conception propres au projet
Synthèse	Kiriku-Wolof-TTS	Seul modèle wolof disponible
Conteneurs	Docker Compose	Parité entre environnements
Intégration	GitHub Actions	Intégration native au dépôt

Table 5.1 – Choix technologiques

5.2 Le moteur de calendrier

Le moteur est le cœur du système. Il répond à une question : étant donné une date de naissance et l'historique des doses reçues, quelles échéances restent à satisfaire, et dans quel état ?

5.2.1 Les cinq statuts

Une échéance peut se trouver dans cinq états.

À venir La date prévue n'est pas atteinte.

Due La date prévue est atteinte ou dépassée de peu ; la dose peut être administrée.

En retard La date prévue est dépassée significativement, mais la dose reste rattrapable.

Périmée La date limite est dépassée : la dose n'est plus administrable selon le schéma.

Administrée Une dose a satisfait l'échéance.

La distinction entre « en retard » et « périmée » n'est pas cosmétique. Elle détermine si un rappel part encore : relancer une mère pour une dose devenue inadministrable serait inutile et, pire, lui ferait faire un déplacement pour rien.

5.2.2 Le recalcul après une dose

Le cas le plus délicat est celui d'une dose administrée en retard. Si un enfant reçoit sa première dose de pentavalent à dix semaines au lieu de six, les doses deux et trois ne peuvent pas conserver leurs dates initiales : l'intervalle minimal de vingt-huit jours ne serait pas respecté.

Le moteur recalcule donc les échéances suivantes à partir de la date *réelle* d'administration, et non de la date théorique.

```
Pour chaque échéance suivante de la même série :
    date_minimale = date_dose_precedente + intervalle_min
    date_theorique = date_naissance + age_cible
    nouvelle_date = max(date_minimale, date_theorique)
```

Listing 5.1 – Principe du recalcul après administration

Ce comportement fait l'objet de neuf cas de test obligatoires, dérivés directement du cahier des charges.

5.3 La composition des messages wolof

Le message adressé à la mère n'est pas une traduction mot à mot du libellé médical. Il a été construit avec un locuteur natif, et les formulations retenues sont consignées dans un document versionné.

5.3.1 Ce que dit le message

Un rappel comporte quatre éléments : une salutation, la désignation du bénéficiaire, le vaccin et ce contre quoi il protège, et l'invitation à venir au poste.

```
Nanga def. Sokhna Ba dafa wara ñakk u ñakk bu lay aar ci juroom i
feebar yu bokkul : diphtérie, tétanos, coqueluche, l'hépatite B ak
infection bu garaw ñu koy woowe Hib tay. Ñëwal ci postu wérgi-yaram bi.
```

Le choix d'énoncer les maladies plutôt que le sigle est délibéré. « Votre enfant doit recevoir le pentavalent » ne dit rien à une mère qui ne connaît pas ce mot. « Le vaccin qui le protège contre cinq maladies » lui dit pourquoi se déplacer.

5.3.2 Deux garde-fous

Cette richesse a un coût : la durée. Un message mentionnant trois vaccins peut atteindre quarante secondes, ce qui est long à écouter.

Deux règles limitent la dérive. Un vaccin n'est jamais répété, même si trois doses sont en retard. Et au-delà de trois vaccins distincts, le message résume au lieu d'énumérer.

5.4 La production du message vocal

5.4.1 Deux mécanismes

La production du son a connu une révision en cours de projet, documentée dans l'ADR 0009.

La conception initiale retenait l'**assemblage de segments pré-enregistrés** : une quarantaine de fragments courts — salutations, noms de vaccins, mois, formules de relance — enregistrés par une voix humaine et concaténés selon le message à produire. Ce mécanisme offre une voix naturelle et ne coûte rien à l'exécution.

Son défaut est apparu à l'usage : il ne sait dire que ce qui a été enregistré. Chaque nouveau poste de santé aurait demandé un enregistrement supplémentaire, ce qui ne tient pas quand le dispositif s'étend.

La disponibilité d'un modèle de **synthèse vocale wolof** a permis de lever cette contrainte. Le modèle Kiriku-Wolof-TTS, d'architecture VITS, publié par AI Hub Sénégal, prononce le texte composé quel qu'il soit.

Les deux mécanismes coexistent : la synthèse est la voie principale, et l'assemblage reste le repli quand le service est indisponible.

5.4.2 Le coût de la synthèse et sa parade

La génération coûte entre cinq et vingt secondes par message sur une machine sans accélérateur graphique. C'est beaucoup trop pour la faire pendant l'envoi.

Deux parades ont été mises en place. La génération se fait *avant* l'envoi, dans une étape distincte du balayage. Et les fichiers sont indexés par l'empreinte du texte, non par le rappel.

Ce second point est ce qui rend la synthèse praticable. Les messages se répètent beaucoup d'une mère à l'autre : deux enfants devant recevoir le même vaccin le même jour produisent un texte identique. Un balayage génère donc une vingtaine de fichiers distincts, et non un par bénéficiaire.

5.5 L'encapsulation vidéo

Un obstacle technique a failli compromettre le projet entier.

WhatsApp n'accepte, en en-tête d'un modèle de message, que du texte, une image, une vidéo ou un document. **L'audio n'en fait pas partie.** Or un message vocal envoyé spontanément, hors de la fenêtre de vingt-quatre heures ouverte par une réponse de la destinataire, n'est pas réalisable directement.

La solution retenue consiste à *encapsuler* l'audio dans une vidéo. Un fichier MP4 est assemblé par `ffmpeg` à partir d'une image fixe aux couleurs de l'application et de la piste audio wolof, puis envoyé en en-tête d'un modèle de catégorie utilitaire.

Le poids reste modeste — de l'ordre de deux cents kilooctets pour vingt secondes — ce qui compte sur un réseau 2G.

5.6 Sécurité

5.6.1 Le cloisonnement par poste

Un agent n'accède qu'aux bénéficiaires de son poste. Cette règle est appliquée au niveau de la requête, non de l'affichage : le jeu de résultats est filtré avant d'être sérialisé, de sorte qu'aucune donnée d'un autre poste ne transite, même vers un client qui la masquerait.

5.6.2 Le mot de passe initial

Un compte créé par un superviseur reçoit un mot de passe choisi par celui-ci et transmis oralement. Tant que son détenteur ne l'a pas changé, l'API refuse toute requête hors du changement lui-même.

Le raisonnement est simple : un mot de passe transmis par un tiers n'est pas un secret. Sans cette règle, un agent travaillerait des mois avec celui que son superviseur connaît.

5.6.3 L'invalidation des sessions

Un changement ou une réinitialisation de mot de passe met en liste noire les jetons de rafraîchissement du compte. Si le mot de passe avait été compromis, les sessions ouvertes ailleurs tombent.

Une limite est assumée : un jeton d'accès reste valable jusqu'à son expiration, soit quinze minutes. C'est inhérent aux jetons sans état, et le prix du choix de ne pas interroger la base à chaque requête.

5.6.4 Le journal d'audit

Chaque acte d'administration — création de compte, réinitialisation, transfert, fermeture de poste — est journalisé avec son auteur, sa cible et son horodatage. Le journal est en écriture seule et réservé à l'administrateur : celui qui est contrôlé ne contrôle pas.

Chapitre 6

Qualité et industrialisation

6.1 Stratégie de tests

La suite de tests compte 286 tests côté serveur et couvre 87% du code. Côté client, la couverture atteint 85,6%.

Niveau	Part	Objet
Unitaires	∼60%	Moteur de calendrier, composition des messages, assemblage audio. Sans base de données.
Intégration	∼35%	API REST, permissions, cloisonnement, webhooks. Avec base de données.
Bout en bout	∼5%	Parcours critiques de l'interface.

Table 6.1 – Répartition des tests

6.1.1 Ce que les tests ont attrapé

Trois défauts méritent d'être cités, parce qu'ils illustrent ce que les tests apportent réellement.

Le rafraîchissement incomplet. La fonction de mise à jour des statuts ne reprenait pas les échéances déjà marquées périmées. Un enfant dont une dose devenait rattrapable après correction de sa date de naissance restait bloqué. Le défaut est apparu en écrivant les tests de rappels avec des dates fixes.

Le cloisonnement du webhook. Un test vérifiant la signature HMAC des appels entrants a révélé que la vérification était contournable en l'absence d'en-tête de signature.

La vidéo non jointe. Le service d'envoi composait bien le message et produisait bien l'audio, mais n'attachait pas la vidéo au message envoyé. Meta acceptait l'envoi puis rejetait silencieusement le message hors fenêtre de vingt-quatre heures. Ce défaut n'avait pas été détecté par les tests, parce que ceux-ci utilisaient le canal de simulation, qui accepte tout — une limite de la stratégie de test qu'il faut reconnaître.

6.2 Intégration continue

Chaque *pull request* déclenche une chaîne de vérifications :

1. Formatage du code (Black, oxlint)
2. Analyse statique (Ruff, TypeScript)
3. Vérification du typage (mypy)
4. Contrôle des migrations en attente
5. Exécution des tests avec seuil de couverture
6. Analyse des dépendances vulnérables

Un échec bloque la fusion. Cette contrainte a été respectée sans exception : les quarante-sept *pull requests* du projet sont toutes passées par cette chaîne.

6.3 Décisions d'architecture

Neuf décisions structurantes ont été consignées sous forme d'ADR.

ADR	Décision
0001	Langue du code : français pour le domaine, anglais pour la technique
0002	Stratégie de branchement et de fusion
0003	Échéances persistées plutôt que recalculées
0004	Idempotence de la synchronisation
0005	Portée de l'internationalisation
0006	Authentification des bénéficiaires
0007	Authentification et administration du personnel
0008	Traçabilité des actes d'administration
0009	Production des messages vocaux (révisée)

Table 6.2 – Décisions d'architecture consignées

L'ADR 0009 est la seule à avoir été révisée en cours de projet. Sa version initiale écartait la synthèse vocale au profit des segments enregistrés, faute de modèle wolof fiable. La disponibilité d'un tel modèle a invalidé la prémisse, et la décision a été reprise. Conserver la trace de ce retournement, plutôt que de réécrire l'histoire, fait partie de l'intérêt de l'exercice.

6.4 Volumétrie

Indicateur	Valeur
Lignes de Python (applications)	∼13 400
Lignes de TypeScript et TSX	∼15 500
Tests serveur	286
Couverture serveur	87%
Couverture client	85,6%
Applications Django	8
Tables en base	18
Décisions d'architecture	9

Table 6.3 – Volumétrie du projet

Chapitre 7

Résultats

Ce chapitre présente le système tel qu'il fonctionne, écran par écran.

7.1 Authentification

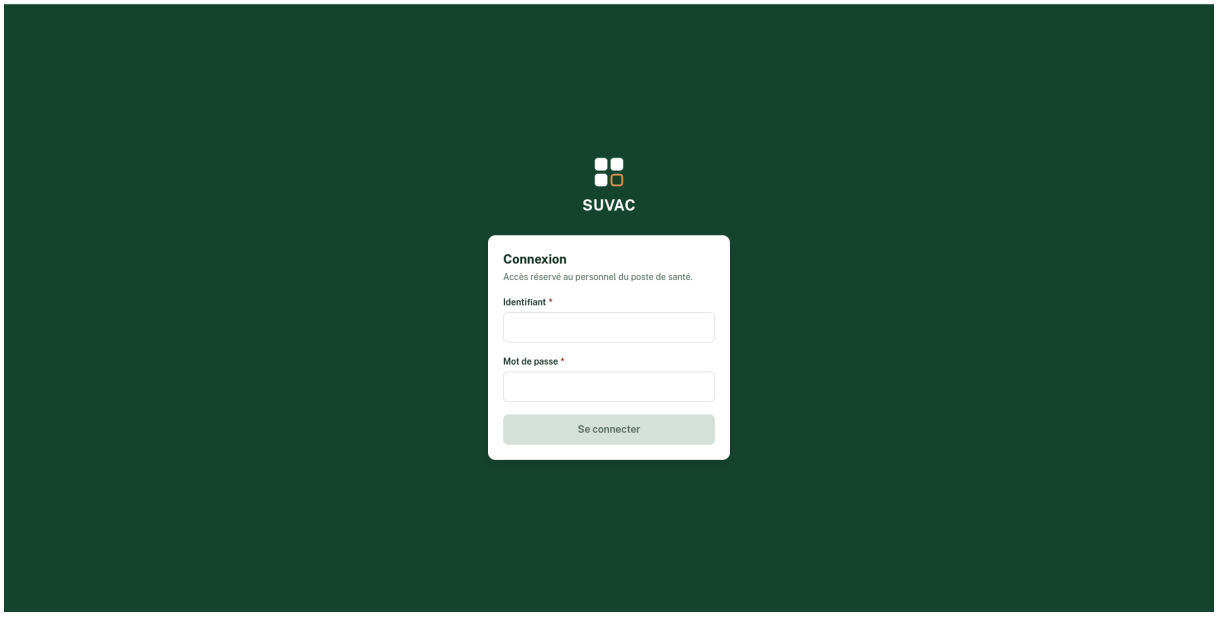


Figure 7.1 – Écran de connexion

7.2 Le poste de travail de l'agent

7.2.1 La file du jour

C'est l'écran d'accueil de l'agent, et la réponse à l'objectif OS-2 : savoir chaque matin qui devrait se présenter.

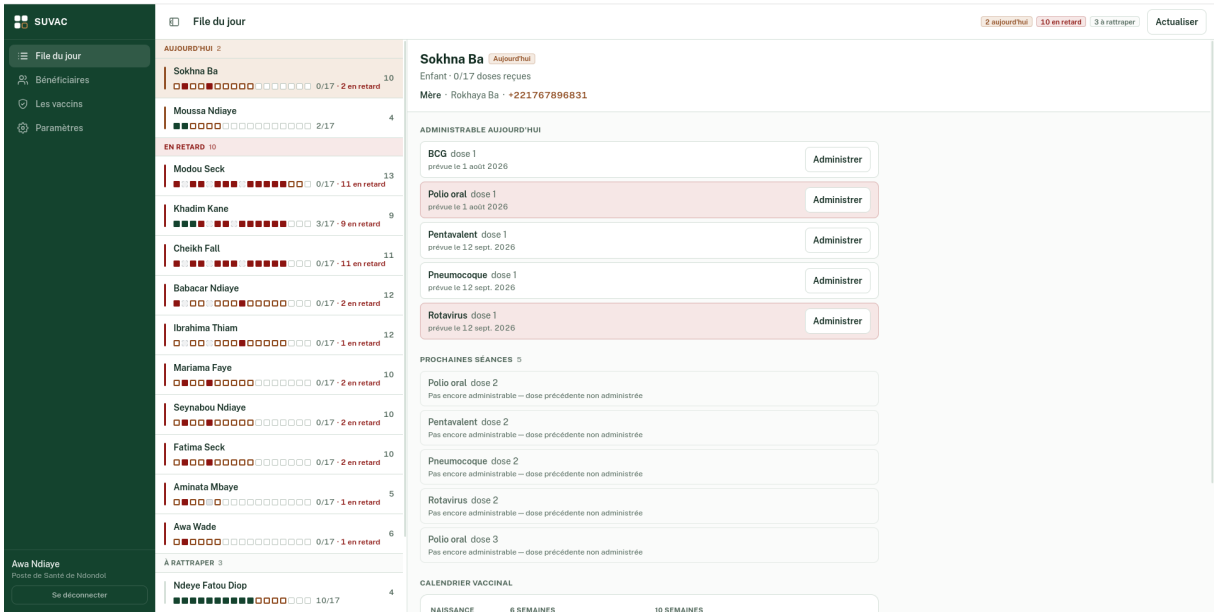


Figure 7.2 – File du jour — bénéficiaires attendus et en retard

Les bénéficiaires sont groupés par urgence. Les doses en retard apparaissent en premier, puisque ce sont elles qui réclament une action.

7.2.2 Le carnet d'un bénéficiaire

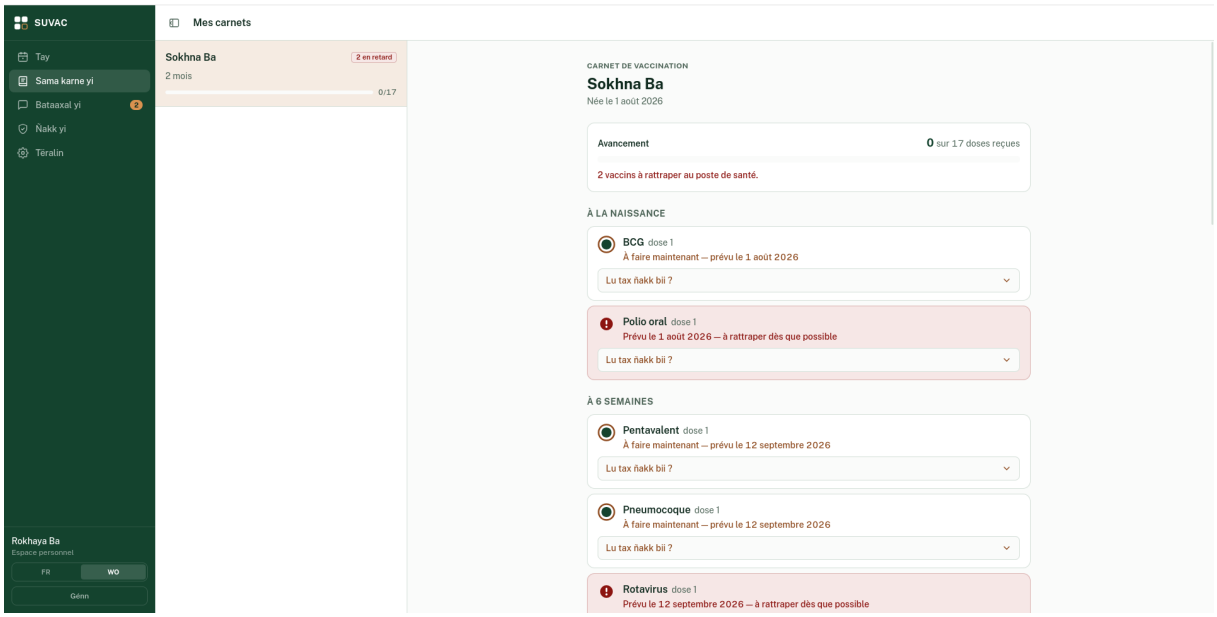


Figure 7.3 – Fiche d'une mère et carnet de ses enfants

La frise verticale montre le parcours vaccinal complet, du passé administré aux échéances à venir. C'est la transposition numérique du carnet papier, avec ce que le papier ne sait pas faire : signaler ce qui est dû.

7.2.3 La saisie d'une dose

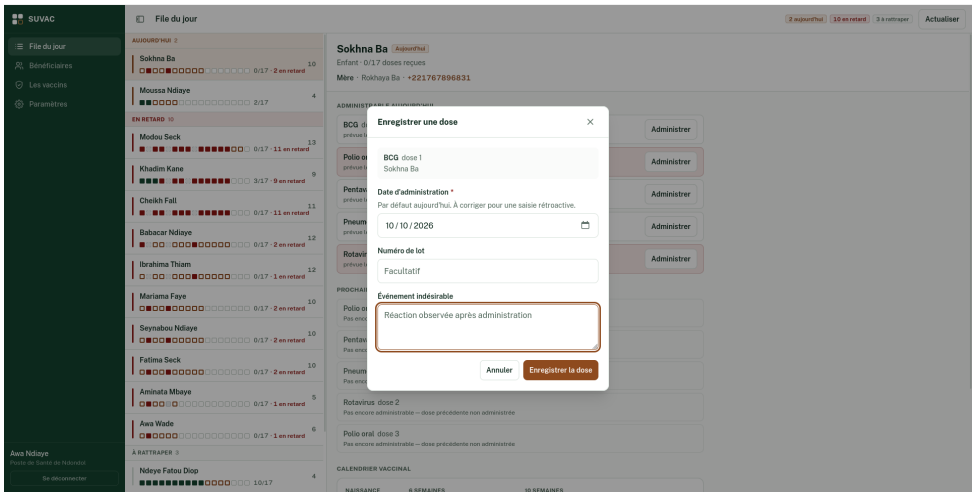


Figure 7.4 – Enregistrement d'une dose administrée

Le formulaire vérifie l'administrabilité avant de permettre l'enregistrement. Une dose dont l'intervalle minimal n'est pas écoulé est refusée avec son motif.

7.3 Le pilotage du superviseur

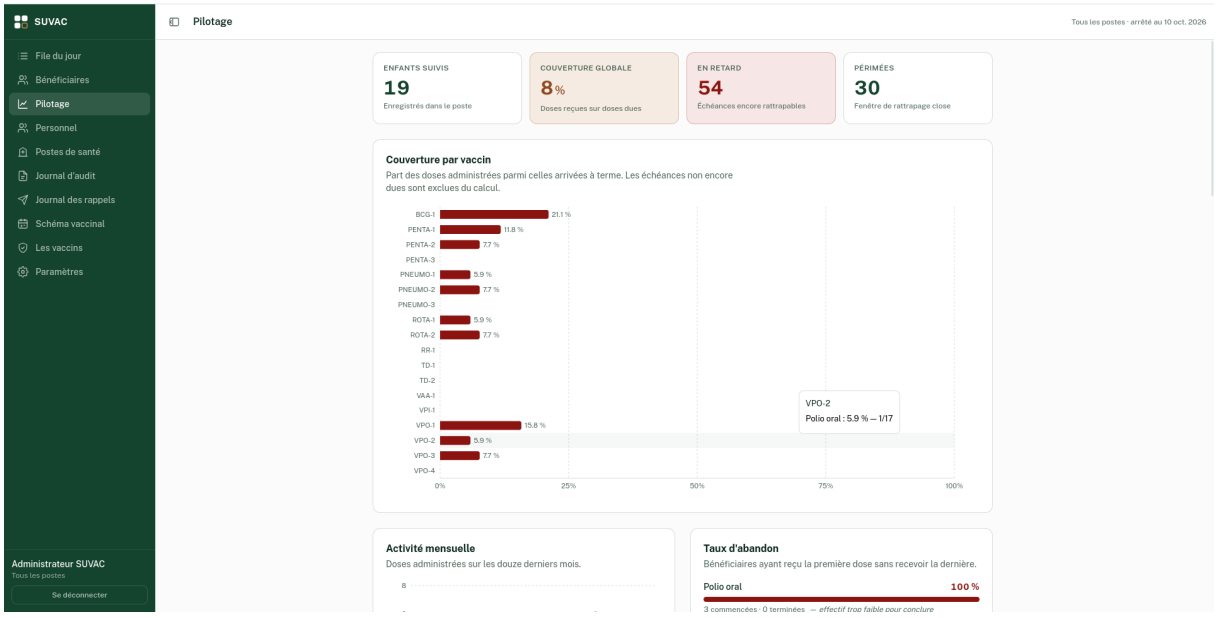


Figure 7.5 – Tableau de bord du superviseur

Les indicateurs de couverture sont calculés sur la cohorte réellement concernée, et non sur l'ensemble des enfants enregistrés. Un enfant de six semaines ne peut pas faire baisser le taux de couverture du vaccin contre la rougeole, qui ne lui est dû qu'à neuf mois.

Cette précision est importante : un dénominateur mal choisi produit des chiffres faux dans le sens rassurant, ce qui est le pire des cas pour un indicateur de santé publique.

7.4 Le journal des rappels

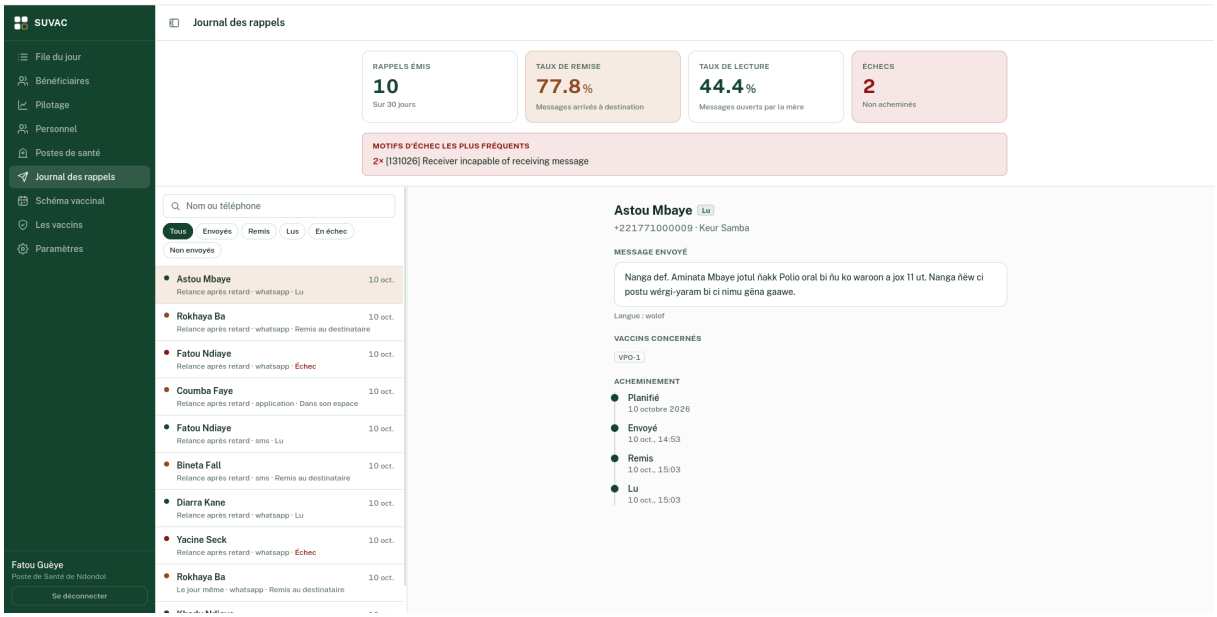


Figure 7.6 – Journal des rappels émis

Le journal donne au superviseur ce dont il a besoin pour savoir si ses messages arrivent : le taux de remise, le taux de lecture, et les motifs d'échec les plus fréquents.

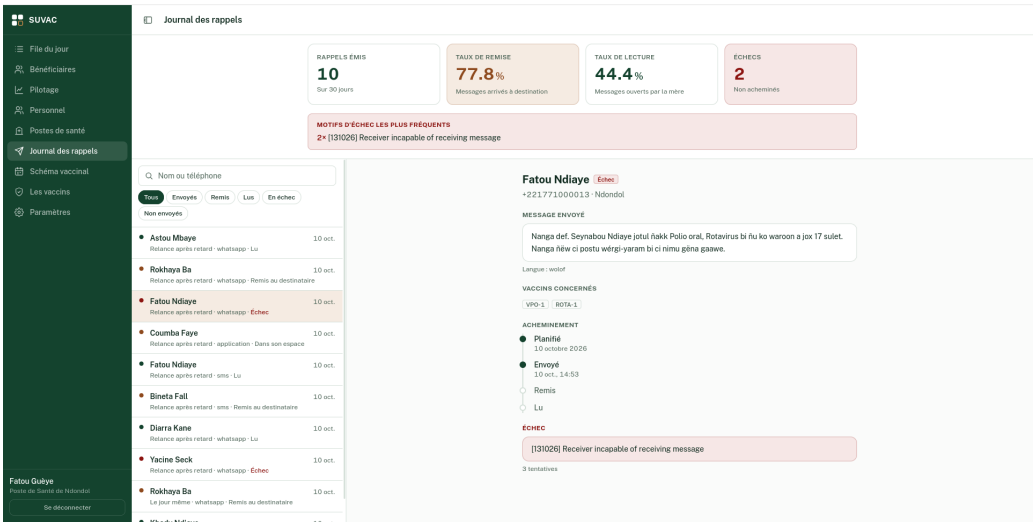


Figure 7.7 – Détail d'un rappel — texte envoyé et frise d'acheminement

Le détail montre le texte wolof exactement tel qu'il a été envoyé. C'est une exigence de traçabilité : en cas de contestation, on doit pouvoir dire ce que la mère a reçu.

7.5 L'espace de la bénéficiaire

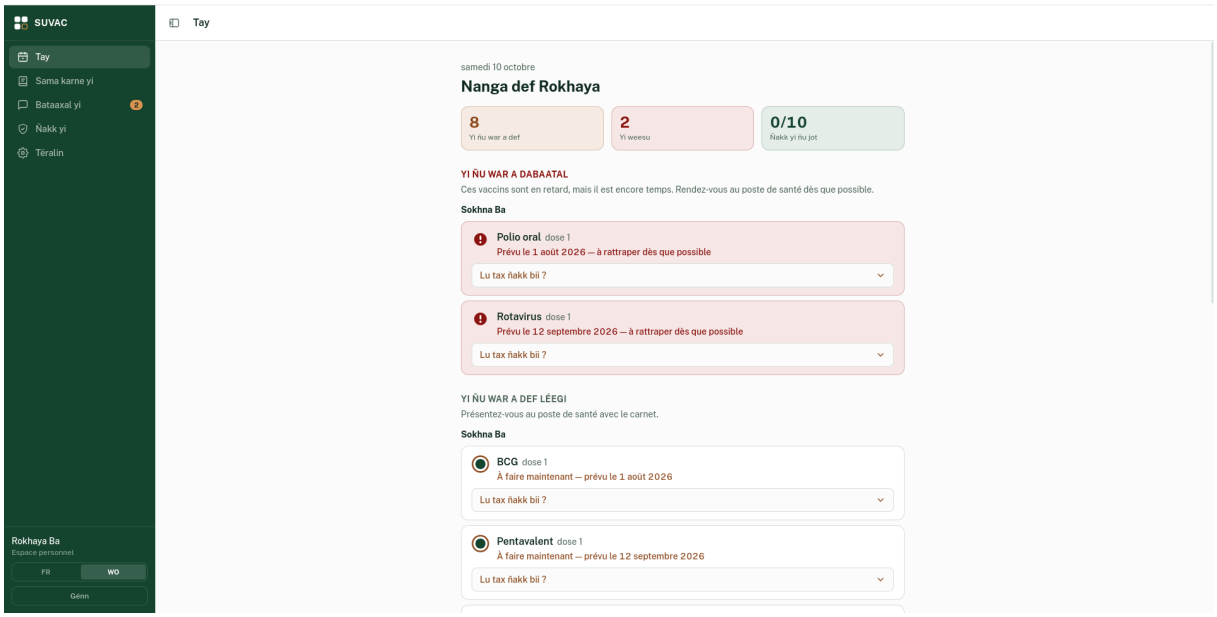


Figure 7.8 – Espace de consultation de la mère

Cet espace est entièrement en wolof. C'est le seul endroit du système où l'interface est traduite : le personnel soignant travaille en français, et traduire son interface aurait été un effort considérable pour un bénéfice nul.

Cette délimitation est documentée dans l'ADR 0005. Elle illustre une leçon du projet : l'internationalisation coûte cher, et il vaut mieux la concentrer là où elle change quelque chose.

7.6 L'administration

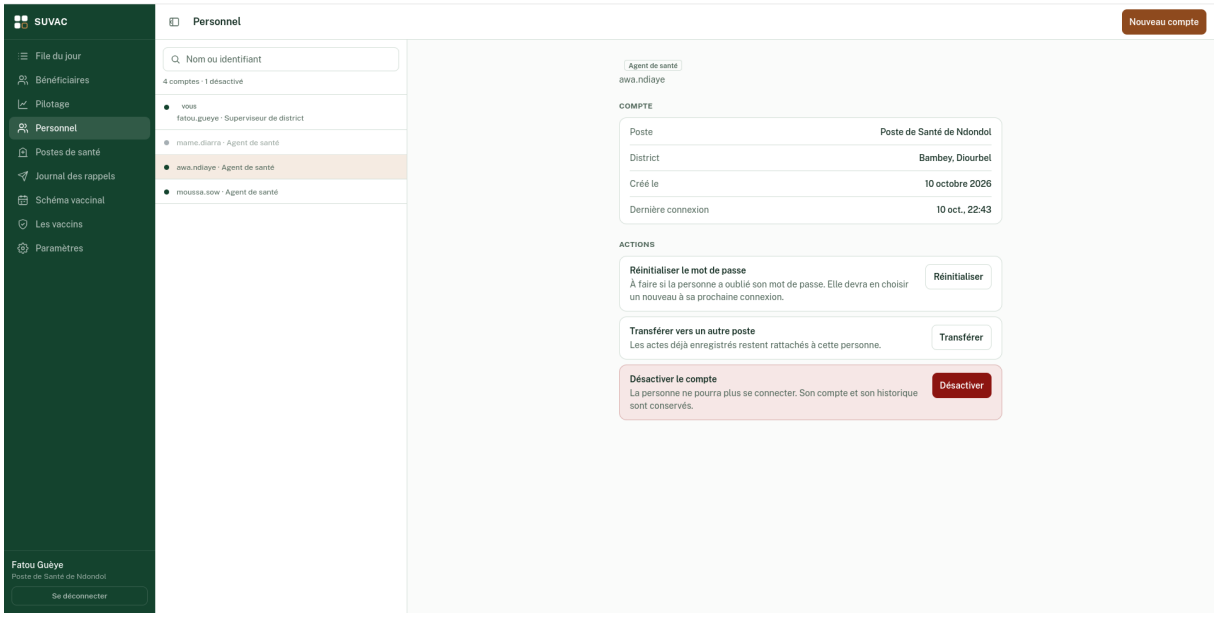


Figure 7.9 – Gestion du personnel du poste

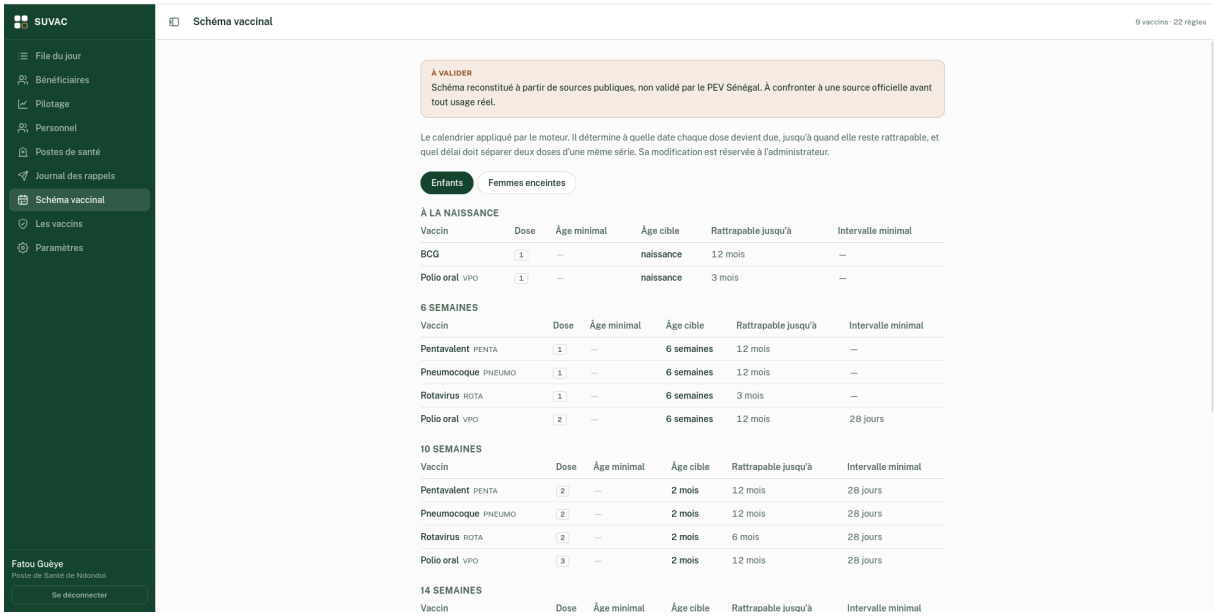


Figure 7.10 – Schéma vaccinal en consultation

Le schéma est consultable mais non modifiable par le superviseur. Une erreur de saisie recalculerait les calendriers de tous les enfants du poste ; cette opération relève de l'administrateur.

L'avertissement affiché en tête d'écran mérite un commentaire. Le schéma utilisé a été reconstitué à partir de sources publiques et n'a pas été validé par le PEV Sénégal. Plutôt que de masquer cette limite, le système l'affiche.

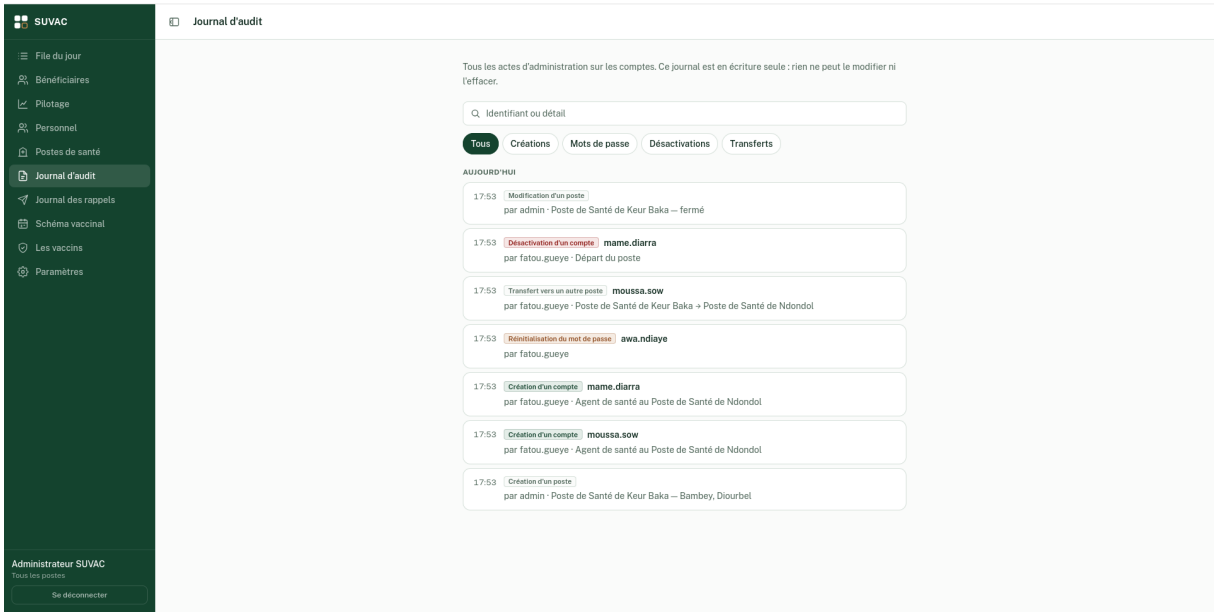


Figure 7.11 – Journal des actes d'administration

7.7 Le rappel reçu

Voici le résultat qui donne son sens au projet : un rappel vaccinal composé en wolof, prononcé par synthèse vocale, encapsulé en vidéo, et reçu sur un téléphone sénégalais.



Figure 7.12 – Rappel vaccinal reçu sur WhatsApp

La vignette porte l'identité de l'application. Le corps du message, en français, nomme l'enfant concerné — il s'adresse à qui sait lire. Le contenu qui compte est dans la vidéo : le message vocal en wolof.

Chapitre 8

Bilan et perspectives

8.1 Ce qui a été livré

Les quatre objectifs spécifiques ont été atteints. Le moteur de calendrier fonctionne et gère les cas difficiles — prématurés, doses en retard, contre-indications. L'agent dispose de sa file quotidienne. Les rappels vocaux partent et arrivent. Le superviseur a ses indicateurs.

Le risque le plus sérieux du projet a été levé : rien ne garantissait, au départ, qu'un message vocal spontané puisse être acheminé par WhatsApp. La réponse — l'encapsulation vidéo — a été trouvée et vérifiée en conditions réelles.

8.2 Ce qui n'a pas été fait

8.2.1 Le mode hors ligne

C'est le manque le plus important. Le cahier des charges spécifie sept exigences relatives au fonctionnement sans connexion, et l'argument de la connectivité dégradée revient dans la justification du projet.

Ces exigences n'ont pas été réalisées. La pièce la plus difficile est toutefois en place depuis le premier sprint : la clé d'idempotence qui permet de rejouer une opération sans la dupliquer. C'est elle qui rend la synchronisation possible sans créer de doses fantômes.

8.2.2 Le code à usage unique

L'authentification des bénéficiaires repose encore sur un mot de passe transmis par l'agent. Le remplacement par un code à usage unique envoyé sur WhatsApp a été conçu mais non livré : il exige un modèle de message d'une catégorie particulière chez Meta, dont l'obtention n'a pas pu être menée à terme.

8.2.3 L'évaluation de la compréhension

Les messages wolof ont été validés par un locuteur natif, et le rendu de la synthèse jugé acceptable à l'écoute. Mais aucune évaluation formelle n'a été conduite auprès de mères wolophones.

C'est la limite la plus sérieuse du projet, et il faut la nommer : le seul critère qui compte est qu'une mère de Ndondol comprenne ce qu'elle entend, et ce critère n'a pas été mesuré.

8.3 Ce que le projet a appris

8.3.1 Les contraintes externes dictent la conception

L'encapsulation vidéo n'est pas une idée élégante : c'est un contournement imposé par une limitation de Meta. De même, le service de synthèse séparé n'existe que parce que deux bibliothèques refusent de cohabiter.

Une part importante du travail d'architecture consiste à absorber ce genre de contraintes sans contaminer le reste du système. L'abstraction des canaux et la séparation du service de synthèse ont servi exactement à cela.

8.3.2 Les tests révèlent ce qu'on ne cherchait pas

Le défaut du rafraîchissement incomplet a été découvert en écrivant des tests de rappels, pas des tests de calendrier. Les tests écrits avec des dates fixes ont exposé une hypothèse implicite sur la date courante.

À l'inverse, la vidéo non jointe est passée au travers, parce que le canal de simulation acceptait tout. Un test qui simule trop bas ne vérifie rien.

8.3.3 Documenter les décisions a une valeur propre

Les neuf ADR ont servi plusieurs fois à retrouver *pourquoi* un choix avait été fait, des semaines après. L'ADR 0009, révisée, montre l'intérêt le plus net : conserver la décision initiale et son renversement donne une information qu'un document réécrit aurait perdue.

8.4 Perspectives

Le mode hors ligne
: Service worker, base locale et file de synchronisation. C'est le chantier prioritaire, et le fondement est en place.

L'évaluation de la compréhension
: Faire écouter des messages à des mères wolophones sans leur montrer le texte, et mesurer ce qu'elles en retiennent.

La validation du schéma vaccinal
: Confronter le référentiel reconstitué à une source officielle du PEV.

Le code à usage unique
: Obtenir le modèle d'authentification chez Meta et supprimer le mot de passe transmis oralement.

Le déploiement en conditions réelles
: Une expérimentation dans un poste de santé volontaire, qui seule permettrait de mesurer l'effet sur les doses manquées.

8.5 Conclusion

SUVAC répond à une question simple : comment prévenir une mère qui ne lit pas que son enfant doit être vacciné.

La réponse technique — un message vocal en wolof acheminé par WhatsApp — a demandé de résoudre plusieurs obstacles qui n'étaient pas prévisibles au départ, du refus de l'audio par Meta à l'incompatibilité des bibliothèques de synthèse. Chacun a été documenté et contourné.

Reste la question qui compte, et qui dépasse le cadre de ce module : est-ce que cela change quelque chose ? Un système qui fonctionne techniquement n'est pas un système qui sert. La réponse exigerait une expérimentation sur le terrain, et c'est la perspective la plus importante de ce travail.

Annexe A

Installation et vérification

A.1 Prise en main rapide

```
git clone https://github.com/Fallou236/suvac.git
cd suvac
docker compose up -d

cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # renseigner DJANGO_SECRET_KEY
python manage.py migrate
python manage.py charger_schema_pev
python manage.py charger_descriptions
python manage.py charger_donnees_demo --vider
python manage.py runserver

# dans un second terminal
cd frontend && npm install && npm run dev
```

L'application est accessible sur `http://localhost:5173`.

A.2 Comptes de démonstration

Mot de passe commun : `suvac-demo-2026`

Identifiant	Rôle	À observer
`awa.ndiaye`	Agent, Ndondol	File du jour, saisie d'une dose
`fatou.gueye`	Superviseur, Ndondol	Pilotage, journal des rappels
`omar.diallo`	Superviseur, Ngaparou	Les mêmes écrans, d'autres chiffres : le cloisonnement
`admin.suvac`	Administrateur	Postes, journal d'audit
`rokhaya.ba`	Bénéficiaire	L'espace de la mère, en wolof

Table A.1 – Comptes du jeu de démonstration

Toutes les données sont fictives, conformément à ENF-24.

A.3 Vérification

```
cd backend && pytest -q          # 286 tests, 87% de couverture
cd frontend && npm run verifier  # analyse, compilation, tests
```

Annexe B

Déclencher un balayage

Pour observer la production des rappels sans attendre l'ordonnanceur :

```
cd backend
python manage.py balayer
```

Sans configuration WhatsApp, les rappels partent en simulation et sont consultables dans le journal avec le compte `fatou.gueye`.

Pour recevoir un rappel réel, attribuer son numéro à une bénéficiaire :

```
python manage.py charger_donnees_demo --vider --telephone "+221771234567"
python manage.py balayer
```

Le numéro doit avoir été déclaré au préalable dans la liste des destinataires autorisés du compte de test Meta.